

As a Senior Backend Architect, I have designed this system with a **microservices-oriented modular monolith** approach. This allows for clear separation of concerns (Clinical, Auth, AI, Ops) while maintaining the operational simplicity required for highly reliable medical environments.

1. Architecture Overview

- **Pattern:** API Gateway + Micro-services (deployed as containers via Kubernetes).
- **Communications:**
 - * **Synchronous:** REST/gRPC for UI-to-Service communication.
 - * **Asynchronous:** RabbitMQ or Kafka for event-driven workflows (e.g., triggering patient notifications, auditing logs).
- **AI Integration:** Sidecar pattern for AI Inference Engines to ensure clinical latency is kept low.

2. Services

1. **Identity Service:** Auth, RBAC, and User Management.
2. **Patient/EHR Service:** CRUD operations for PHI; strict data isolation.
3. **Scheduling Service:** Handles appointments, waitlists, and triage logic.
4. **AI Orchestrator Service:** Manages LLM/ML model calls, prompt injection protection, and "Human-in-the-loop" state management.
5. **Resource Service:** Real-time occupancy tracking for beds/assets.
6. **Billing/Insurance Service:** Financial transaction ledger.

3. Modules

- **Audit Module:** Generic service for capturing immutable event streams.
- **Notification Module:** Multi-channel (SMS, Email, Push) triggered via events.
- **Security Module:** Middleware for HIPAA compliance, TLS 1.3, and PII masking.

4. Authentication & Authorization

- **Auth Provider:** OpenID Connect (OIDC) with Auth0 or Keycloak for centralized management.
- **Authorization:** Attribute-Based Access Control (ABAC). It is more granular than RBAC for HIPAA, allowing rules like: "Doctor can only access EHR if they are the assigned provider OR the patient is currently checked in."

- **Tokens:** Short-lived JWTs with refresh token rotation.

5. Business Logic

- **Clinical Guardrails:** Business logic is implemented in a "Domain Layer" that is agnostic of the DB.
- **Human-in-the-Loop:** All AI requests return a ``pending_verification`` state. The UI remains disabled for that action until a ``PATCH`` request with a clinician's signature (JWT-authenticated) is received.

6. Background Jobs

- **Engine:** Temporal.io or BullMQ.
- **Use Cases:**
 - * Generating daily resource reports for Admins.
 - * Cleaning up expired appointment slots.
 - * Asynchronous triggering of AI model re-training pipelines.

7. File Storage

- **Solution:** S3-compatible storage (e.g., MinIO for on-prem HIPAA compliance).
- **Security:** Objects are encrypted at rest using per-client keys (KMS). Medical images (DICOM) are stored as encrypted blobs with metadata linked in the EHR table.

8. Error Handling & Logging

- **Global Error Handler:** Centralized middleware to map domain exceptions to specific HTTP status codes (e.g., ``PatientNotFoundError`` -> 404).
- **Structured Logging:** ELK Stack (Elasticsearch, Logstash, Kibana) or Grafana Loki. All logs must contain ``request_id`` and ``user_id`` for traceability.

9. Recommended Folder Structure

```
/src
  /config          # Environment/Compliance config
  /domain          # Core business entities
  /interfaces      # API endpoints (Controllers/GraphQL resolvers)
  /application     # Use-case logic (Services)
  /infrastructure  # Adapters (DB queries, S3 clients, AI models)
  /shared          # Utilities, Logging, Error handling
/tests
  /unit           # Business logic tests
  /integration    # API & DB contract tests
```

10. Recommended Libraries (Node.js/TypeScript stack)

- **API Framework:** `Fastify` (High performance/Schema validation via JSON Schema).
- **ORM:** `Prisma` or `Drizzle` (Type-safe migrations and queries).
- **Validation:** `Zod` (Runtime data validation for request bodies).
- **Messaging:** `BullMQ` (Redis-backed queues for background jobs).
- **AI:** `LangChain.js` (For managing prompt chains and AI memory).
- **Testing:** `Vitest` + `Supertest` (For performance and speed).
- **Documentation:** `Swagger/OpenAPI` (Automated via Fastify Swagger).

Key Architectural Strategy: "Compliance as Code"

To meet **NFR1 (Security)** and the business rule regarding **Human-in-the-loop**, we will implement a **Database Trigger/Middleware wrapper**. Any write operation to `EHR\_Records` that includes an `ai\_suggestion\_id` will be rejected by the API Gateway unless the payload includes a valid digital signature of a doctor, ensuring that the system is literally incapable of bypassing the oversight requirement.